下载APP

10 | 生命周期：你创建的值究竟能活多久？

2021-09-13 陈天

《陈天 · Rust 编程第一课》 课程介绍 >



讲述：陈天
时长 14:07 大小 12.94M

你好，我是陈天。

之前提到过，在任何语言里，栈上的值都有自己的生命周期，它和帧的生命周期一致，而 Rust，进一步明确这个概念，并且为堆上的内存也引入了生命周期。

我们知道，在其它语言中，堆内存的生命周期是不确定的，或者是未定义的。因此，要么开发者手工维护，要么语言在运行时做额外的检查。而在 Rust 中，除非显式地做 Box::leak() / Box::into_raw() / ManualDrop 等动作，**一般来说，堆内存的生命周期，会默认和其栈内存的生命周期绑定在一起。**

所以在这种默认情况下，在每个函数的作用域中，编译器就可以对比值和其引用的生命周期，来确保“引用的生命周期不超出值的生命周期”。

那你有没有想过，Rust 编译器是如何做到这一点的呢？

值的生命周期

在进一步讨论之前，我们先给值可能的生命周期下个定义。

如果一个值的生命周期**贯穿整个进程的生命周期**，那么我们就称这种生命周期为**静态生命周期**。

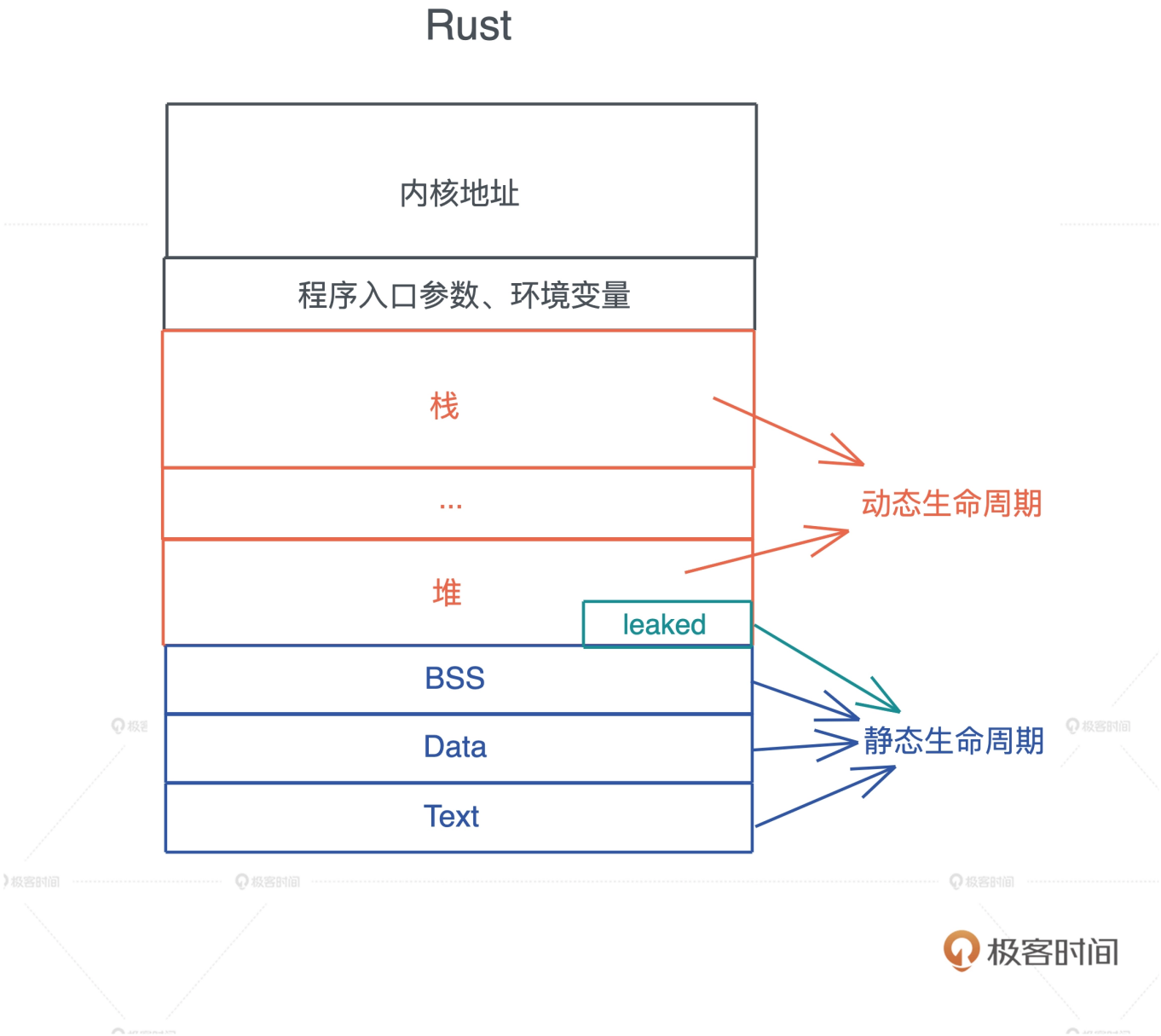
当值拥有静态生命周期，其引用也具有静态生命周期。我们在表述这种引用的时候，可以用 `'static` 来表示。比如： `&'static str` 代表这是一个具有静态生命周期的字符串引用。

一般来说，全局变量、静态变量、字符串字面量（string literal）等，都拥有静态生命周期。我们上文中提到的堆内存，如果使用了 `Box::leak` 后，也具有静态生命周期。

如果一个值是**在某个作用域中定义的**，也就是说它被创建在栈上或者堆上，那么其**生命周期是动态的**。

当这个值的作用域结束时，值的生命周期也随之结束。对于动态生命周期，我们约定用 `'a`、`'b` 或者 `'hello` 这样的小写字符或者字符串来表述。`'` 后面具体是什么名字不重要，它代表某一段动态的生命周期，其中， `&'a str` 和 `&'b str` 表示这两个字符串引用的生命周期可能不一致。

我们通过图总结一下：



分配在堆和栈上的内存有其各自的作用域，它们的生命周期是动态的。

全局变量、静态变量、字符串字面量、代码等内容，在编译时，会被编译到可执行文件中的 BSS/Data/RoData/Text 段，然后在加载时，装入内存。因而，它们的生命周期和进程的生命周期一致，所以是静态的。

所以，函数指针的生命周期也是静态的，因为函数在 Text 段中，只要进程活着，其内存一直存在。

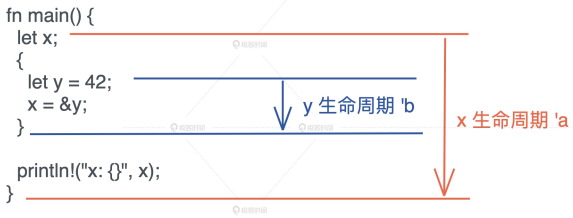
明白了这些基本概念后，我们来看对于值和引用，编译器是如何识别其生命周期的。

编译器如何识别生命周期

我们先从两个最基本最简单的例子开始。

左图的例 1 ，x 引用了在内层作用域中创建出来的变量 y。由于，变量从开始定义到其作用域结束的这段时间，是它的生命周期，所以 x 的生命周期 'a 大于 y 的生命周期 'b，当 x 引用 y 时，编译器报错。

右图例 2 中，y 和 x 处在同一个作用域下， x 引用了 y，我们可以看到 x 的生命周期 'a 和 y 的生命周期 'b 几乎同时结束，或者说 'a 小于等于 'b，所以，x 引用 y 是可行的。



例 1：'a > 'b, x（y 的引用）生命周期大于 y



例 2：'a <= 'b, x（y 的引用）生命周期等于 y

极客时间

这两个小例子很好理解，我们再看个稍微复杂一些的。

示例代码在 main() 函数里创建了两个 String，然后将其传入 max() 函数比较大小。max() 函数接受两个字符串引用，返回其中较大的那个字符串的引用（示例代码）：

复制代码

```
fn main() {
    let s1 = String::from("Lindsey");
    let s2 = String::from("Rosie");

    let result = max(&s1, &s2);

    println!("bigger one: {}", result);
}

fn max(s1: &str, s2: &str) -> &str {
    if s1 > s2 {
        s1
    } else {
        s2
    }
}
```

这段代码是无法编译通过的，它会报错 “missing lifetime specifier” ，也就是说，**编译器在编译 max() 函数时，无法判断 s1、s2 和返回值的生命周期。**

你是不是很疑惑，站在我们开发者的角度，这个代码理解起来非常直观，在 main() 函数里 s1 和 s2 两个值生命周期一致，它们的引用传给 max() 函数之后，无论谁的被返回，生命周期都不会超过 s1 或 s2。所以这应该是一段正确的代码啊？

为什么编译器报错了，不允许它编译通过呢？我们把这段代码稍微扩展一下，你就能明白编译器的困惑了。

在刚才的示例代码中，我们创建一个新的函数 get_max()，它接受一个字符串引用，然后和 “Cynthia” 这个字符串字面量比较大小。之前我们提到，**字符串字面量的生命周期是静态的，而 s1 是动态的，它们的生命周期显然不一致**（ 代码 ）：

复制代码

```
fn main() {
    let s1 = String::from("Lindsey");
    let s2 = String::from("Rosie");

    let result = max(&s1, &s2);

    println!("bigger one: {}", result);

    let result = get_max(s1);
    println!("bigger one: {}", result);
}

fn get_max(s1: &str) -> &str {
    max(s1, "Cynthia")
}

fn max(s1: &str, s2: &str) -> &str {
    if s1 > s2 {
        s1
    } else {
        s2
    }
}
```

当出现了多个参数，它们的生命周期可能不一致时，返回值的生命周期就不好确定了。编译器在编译某个函数时，并不知道这个函数将来有谁调用、怎么调用，所以，**函数本身携带的信息，就是编译器在编译时使用的全部信息。**

根据这一点，我们再看示例代码，在编译 max() 函数时，参数 s1 和 s2 的生命周期是什么关系、返回值和参数的生命周期又有什么关系，编译器是无法确定的。

此时，就需要我们在函数签名中提供生命周期的信息，也就是生命周期标注（lifetime specifier）。在生命周期标注时，使用的参数叫生命周期参数（lifetime parameter）。通过生命周期标注，我们告诉编译器这些引用间生命周期的约束。

生命周期参数的描述方式和泛型参数一致，不过只使用小写字母。这里，两个入参 s1、s2，以及返回值都用 `'a` 来约束。**生命周期参数，描述的是参数和参数之间、参数和返回值之间的关系，并不改变原有的生命周期。**

在我们添加了生命周期参数后，s1 和 s2 的生命周期只要大于等于（outlive）`'a`，就符合参数的约束，而返回值的生命周期同理，也需要大于等于 `'a`。

在你运行上述示例代码的时候，编译器已经提示你，可以这么修改 max() 函数：

复制代码

```
fn max<'a>(s1: &'a str, s2: &'a str) -> &'a str {
    if s1 > s2 {
        s1
    } else {
        s2
    }
}
```

当 main() 函数调用 max() 函数时，s1 和 s2 有相同的生命周期 `'a`，所以它满足 `（s1: &'a str, s2: &'a str）` 的约束。当 get_max() 函数调用 max() 时，“Cynthia” 是静态生命周期，它大于 s1 的生命周期`'a`，所以它也可以满足 max() 的约束需求。

你的引用需要额外标注吗

学到这里，你可能会有困惑了：为什么我之前写的代码，很多函数的参数或者返回值都使用了引用，编译器却没有提示我要额外标注生命周期呢？

这是因为编译器希望尽可能减轻开发者的负担，其实所有使用了引用的函数，都需要生命周期的标注，只不过编译器会自动做这件事，省却了开发者的麻烦。

比如这个例子，first() 函数接受一个字符串引用，找到其中的第一个单词并返回（代码）：

复制代码

```
fn main() {
    let s1 = "Hello world";

    println!("first word of s1: {}", first(&s1));
}

fn first(s: &str) -> &str {
    let trimmed = s.trim();
    match trimmed.find(' ') {
        None => "",
        Some(pos) => &trimmed[..pos],
    }
}
```

虽然我们没有做任何生命周期的标注，但编译器会通过一些简单的规则为函数自动添加标注：

1. 所有引用类型的参数都有独立的生命周期 'a 、'b 等。
2. 如果只有一个引用型输入，它的生命周期会赋给所有输出。
3. 如果有多个引用类型的参数，其中一个是 self，那么它的生命周期会赋给所有输出。

规则 3 适用于 trait 或者自定义数据类型，我们先放在一边，以后遇到会再详细讲的。例子中的 first() 函数通过规则 1 和 2，可以得到一个带生命周期的版本（代码）：

复制代码

```
fn first<'a>(s: &'a str) -> &'a str {
    let trimmed = s.trim();
    match trimmed.find(' ') {
```

```
        None => "",
        Some(pos) => &trimmed[..pos],
    }
}
```

你可以看到，所有引用都能正常标注，没有冲突。那么对比之前返回较大字符串的示例代码（示例代码），max() 函数为什么编译器无法处理呢？

按照规则 1，我们可以对 max() 函数的参数 s1 和 s2 分别标注'a 和'b，但是**返回值如何标注？是 'a 还是'b 呢？这里的冲突，编译器无能为力**。

复制代码

```
fn max<'a, 'b>(s1: &'a str, s2: &'b str) -> &'??? str
```

所以，只有我们明白了代码逻辑，才能正确标注参数和返回值的约束关系，顺利编译通过。

引用标注小练习

好，Rust 的生命周期这个知识点我们就讲完了，接下来我们来尝试写一个字符串分割函数 strtok()，即时练习一下，如何加引用标注。

相信有过 C/C++ 经验的开发者都接触过这个 strtok() 函数，它会把字符串按照分隔符（delimiter）切出一个 token 并返回，然后将传入的字符串引用指向后续的 token。

用 Rust 实现并不困难，由于传入的 s 需要可变的引用，所以它是一个指向字符串引用的可变引用 `&mut &str`（练习代码）：

复制代码

```
pub fn strtok(s: &mut &str, delimiter: char) -> &str {
    if let Some(i) = s.find(delimiter) {
        let prefix = &s[..i];
        // 由于 delimiter 可以是 utf8，所以我们需要获得其 utf8 长度，
        // 直接使用 len 返回的是字节长度，会有问题
        let suffix = &s[(i + delimiter.len_utf8())..];
        *s = suffix;
        prefix
    } else { // 如果没找到，返回整个字符串，把原字符串指针 s 指向空串
```

```
        let prefix = *s;
        *s = "";
        prefix
    }
}

fn main() {
    let s = "hello world".to_owned();
    let mut s1 = s.as_str();
    let hello = strtok(&mut s1, ' ');
    println!("hello is: {}, s1: {}, s: {}", hello, s1, s);
}
```

当我们尝试运行这段代码时，会遇到生命周期相关的编译错误。类似刚才讲的示例代码，是因为按照编译器的规则， `&mut &str` 添加生命周期后变成 `&'b mut &'a str`，这将导致返回的 `'&str` 无法选择一个合适的生命周期。

要解决这个问题，我们首先要思考一下：返回值和谁的生命周期有关？是指向字符串引用的可变引用 &mut ，还是字符串引用 &str 本身？

显然是后者。所以，我们可以为 strtok 添加生命周期标注：

复制代码

```
pub fn strtok<'b, 'a>(s: &'b mut &'a str, delimiter: char) -> &'a str {...}
```

因为返回值的生命周期跟字符串引用有关，我们只为这部分的约束添加标注就可以了，剩下的标注交给编译器自动添加，所以代码也可以简化成如下这样，让编译器将其扩展成上面的形式：

复制代码

```
pub fn strtok<'a>(s: &mut &'a str, delimiter: char) -> &'a str {...}
```

最终，正常工作的代码如下（练习代码 _ 改），可以通过编译：

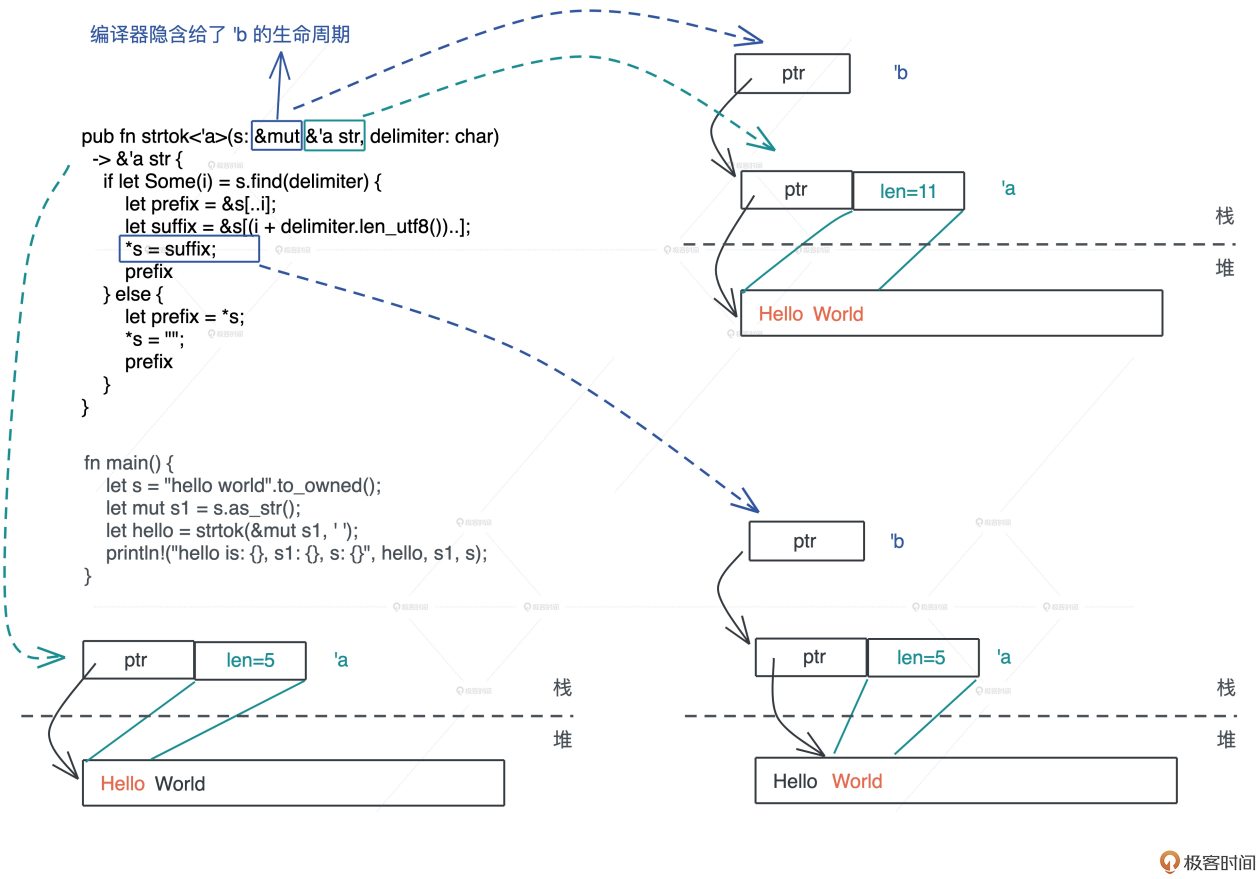
复制代码

```
pub fn strtok<'a>(s: &mut &'a str, delimiter: char) -> &'a str {
    if let Some(i) = s.find(delimiter) {
        let prefix = &s[..i];
```

```
        let suffix = &s[(i + delimiter.len_utf8())..];
        *s = suffix;
        prefix
    } else {
        let prefix = *s;
        *s = "";
        prefix
    }
}

fn main() {
    let s = "hello world".to_owned();
    let mut s1 = s.as_str();
    let hello = strtok(&mut s1, ' ');
    println!("hello is: {}, s1: {}, s: {}", hello, s1, s);
}
```

为了帮助你更好地理解这个函数的生命周期关系，我将每个堆上和栈上变量的关系画了个图供你参考。



这里跟你分享一个小技巧：如果你觉得某段代码理解或者分析起来很困难，也可以画类似的图，从最基础的数据在堆和栈上的关系开始想，就很容易厘清脉络。

在处理生命周期时，编译器会根据一定规则自动添加生命周期的标注。然而，当自动标注产生冲突时，需要我们手工标注。

生命周期标注的目的是，在参数和返回值之间建立联系或者约束。调用函数时，传入的参数的生命周期需要大于等于（outlive）标注的生命周期。

当每个函数都添加好生命周期标注后，编译器，就可以从函数调用的上下文中分析出，在传参时，引用的生命周期，是否和函数签名中要求的生命周期匹配。如果不匹配，就违背了“引用的生命周期不能超出值的生命周期”，编译器就会报错。

如果你搞懂了函数的生命周期标注，那么数据结构的生命周期标注也是类似。比如下面的例子，Employee 的 name 和 title 是两个字符串引用，Employee 的生命周期不能大于它们，否则会访问失效的内存，因而我们需要妥善标注：

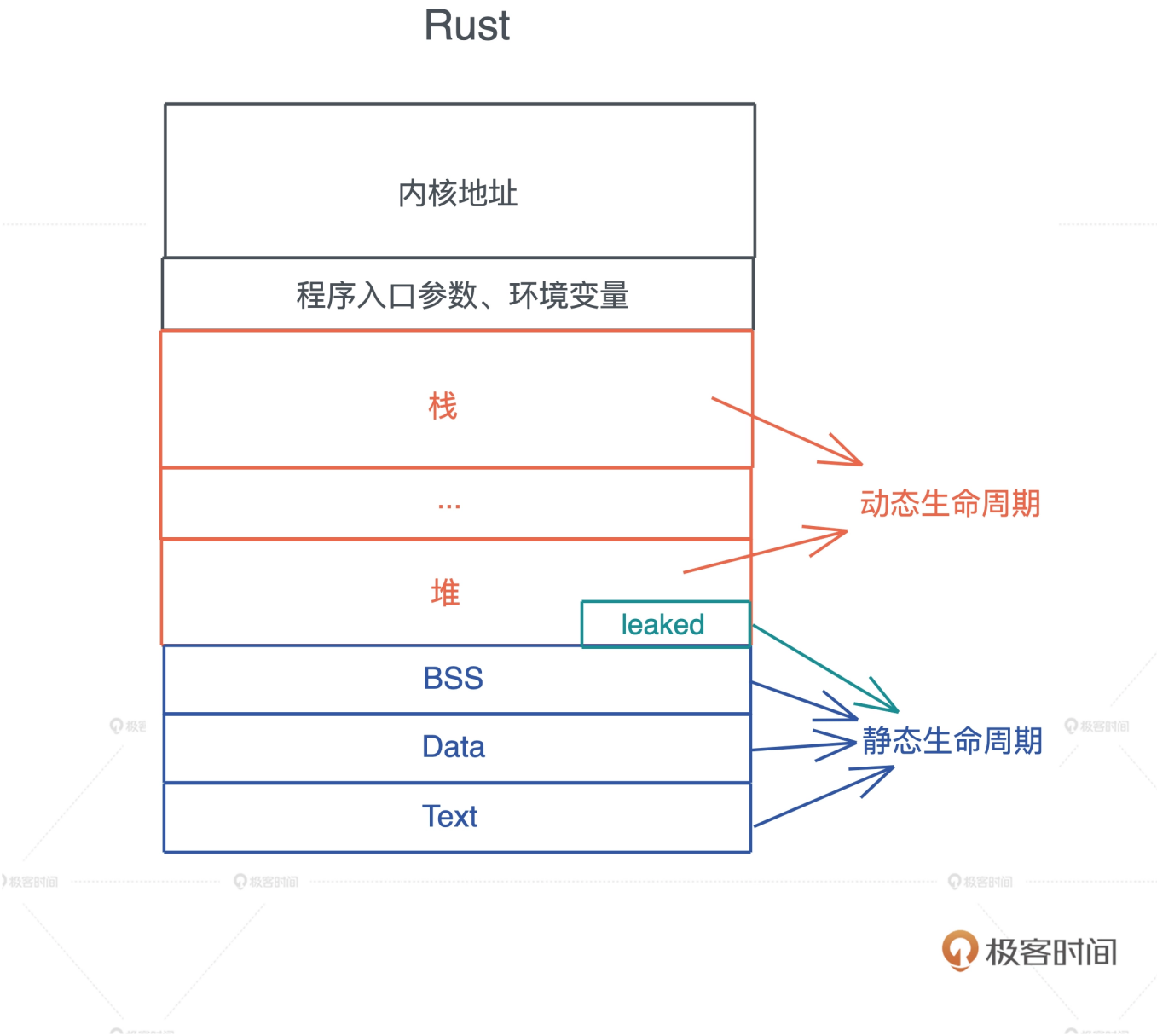
复制代码

```
struct Employee<'a, 'b> {
  name: &'a str,
  title: &'b str,
  age: u8,
}
```

使用数据结构时，数据结构自身的生命周期，需要小于等于其内部字段的所有引用的生命周期。

小结

今天我们介绍了静态生命周期和动态生命周期的概念，以及编译器如何识别值和引用的生命周期。



根据所有权规则，值的生命周期可以确认，它可以一直存活到所有者离开作用域；而引用的生命周期不能超过值的生命周期。在同一个作用域下，这是显而易见的。然而，**当发生函数调用时，编译器需要通过函数的签名来确定，参数和返回值之间生命周期的约束**。

大多数情况下，编译器可以通过上下文中的规则，自动添加生命周期的约束。如果无法自动添加，则需要开发者手工来添加约束。一般，我们只需要确定好返回值和哪个参数的生命周期相关就可以了。而对于数据结构，当内部有引用时，我们需要为引用标注生命周期。

思考题

1. 如果我们把 strtok() 函数的签名写成这样，会发生什么问题？为什么它会发生这个问题？你可以试着编译一下看看。

复制代码

```
pub fn strtok<'a>(s: &'a mut &str, delimiter: char) -> &'a str {...}
```

2. 回顾第 6 讲 SQL 查询工具的代码，现在，看看你是不是对代码中的生命周期标注有了更深理解？

感谢你的收听，你已经打卡 Rust 学习 10 次啦！

如果你觉得有收获，也欢迎你分享给你身边的朋友，邀他一起讨论。坚持学习，我们下节课见。

参考资料

1. 栈上的内存不必特意释放，顶多是编译时编译器不再允许该变量被访问。因为栈上的内存会随着栈帧的结束而结束。如果你有点模糊，可以再看看前置知识，温习一下栈和堆。
2. Rust 的 I/O 安全性目前是 “almost safety” ，为什么不是完全安全，感兴趣的同学可以看这个 RFC。
3. 更多关于 Box::leak 的信息。
4. ArcInner 的结构。
5. Rust 的生命周期管理一直在进化，进化方向是在常见的场景下，尽量避免因为生命周期的处理，代码不得不换成不那么容易阅读的写作方式。比如下面的代码：

复制代码

```
use std::collections::HashMap;

fn main() {
    let mut map = HashMap::new();
    map.insert("hello", "world");
    let key = "hello1";

    // 按照之前的说法，这段代码无法编译通过，因为同一个 scope 下不能有两个可变引用
    // 但因为 RFC2094 non-lexical lifetimes，Rust 编译器可以处理这个场景，
```

```
    // 因为当 None 时，map.get_mut() 的引用实际已经结束
    match map.get_mut(key) /* <----- 可变引用的生命周期一直持续到 match 结果 */ {
        Some(v) => do_something(v),
        None => {
            map.insert(key, "tyr"); // <--- 这里又获得了一个可变引用
        }
    }
}

fn do_something(_v: &mut &str) {
    todo!()
}
```

如果你对此感兴趣，想了解更多，可以参看：RFC2094 - Non-lexical lifetimes。我们在平时写代码时，可以就像这段代码这样先按照正常的方式去写，如果编译器抱怨，再分析引用的生命周期，换个写法。此外，随时保持你的 Rust 版本是最新的，也有助于让你的代码总是可以使用最简单的方式撰写。

分享给需要的人，Ta订阅后你可得 20 元现金奖励

赞 9 提建议

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 09 | 所有权：一个值可以有多个所有者么？

下一篇 11 | 内存管理：从创建到消亡，值都经历了什么？

精选留言 (15)

写留言



记事本 置顶

2021-09-14

&str 是静态区域内存的一个指针
String 是把字符分配在堆上，但是也可以通过as_str以指针形式返回，就是&str类型了，数据类型相同，分配的区域不同，生命周期不同,这样说法对吗？

展开

作者回复: &str 不是静态区域内存的指针。&str 是一个字符串切片，一个带有长度的胖指针，指向字符串实际的位置。它可以指向 "hello world"，此时指针指到了 RODATA/STRING section 里 "hello" 的地址，它的生命周期是 'static；也可以指向 "hello world".to_string()，此时指针指向了这个字符串的堆地址，生命周期是 'a。



Marvichov
2021-09-13

1. 入参 s: &'a mut &str 变成了 &'a mut &'a str; 因为outer ref must outlive inter ref; 相当于把mutable borrow的lifetime extend成了&str的lifetime; 很奇怪, 就算如此, 入参`s` 在函数call结束之后就end its life; 为啥还能extend it life到函数invocation后呢? 希望老师解答一下...

展开

作者回复: 1. 好问题。还是要从生命周期中参数和返回值的关系来看，因为现在我们把对字符串的可变引用的生命周期和返回值的生命周期关联起来了，hello 得到了 strtok 的返回值，所以从编译器看来，&mut s1 的生命周期并没有结束，所以发生了 &mut 和 & 同时存在的局面。
2. 👍
3. 赞
4. 我觉得这是个翻译风格问题，从字面意思上看生命期更好，但大家最惯用的说法已经约定俗称，而且不光 Rust，其它语言在谈到 lifetime 时也是如此（比如 react），所以生命周期更好。它起码比福尔摩斯的译名要好吧 lol。
5. 不会。生命周期在单体化之前就会被处理掉。它更多以编译器的错误呈现出来。对编译器来说，只要满足约束，更长的生命周期的变量也可以通过编译（比如 &'static str 至于 &'a str）。如果生命周期做单体化，下面的代码就可以编译，然而不行：

```rust
trait Print {
    fn print(self);
}

// 生命周期无法像泛型那样有多个实现
// impl<'a> Print for &'a str {
// fn print(self) {
// println!("Arbitrary str: {}", self);
// }
// }
```

```
impl Print for &'static str {
    fn print(self) {
        println!("'static str: {}", self);
    }
}

// 也无法单体化
fn print_str<'a>(s: &'a str) {
    s.print()
}

fn main() {
    let s = "hello, world!";
    s.print();
    print_str(s);
}
```

playground: https://play.rust-lang.org/?version=stable&mode=debug&edition=2018&gist=b767ef1a9d9e265252c1ad29d31176b0

2 7



CR
2021-09-13

想成为今天第一个打卡的男孩 LOL

展开

作者回复: 👍

2



周烨
2021-09-13

1. 错误是 cannot borrow `s1` as immutable because it is also borrowed as mutable。感觉应该是这样写，就变成了可变借用&mut的生命周期和返回值hello的生命周期一样了，所以这个引用的生命周期在函数结束后仍没有结束，所以产生了作用域冲突。我之前加生命周期的时候第一直觉也是加在这个地方，感觉真的很容易出错。

展开

作者回复: 对！

的确，在不是特别理解 &mut &str 在内存中的意义时容易出错，我们需要知道它是一个指向字符串引用的可变指针。这样，返回值和谁产生约束就一目了然了。

1 1



芥末小龙
2021-09-13

陈老师您好！

```
fn max<'a,'b:'a>(s1:&'a str, s2:&'b str) ->&'a str {
    if s1 > s2 {
        s1
    } else {...
```

展开

作者回复: 当 Rust 编译器不能标注生命周期需要开发者进行标注时，一定是编译器无法从已知的逻辑中找到参数和返回值之间的关系。所以我们要说明这种关系。

我们可以说 s1, s2 生命周期一致，返回值和他们的一致。这是我文中代码的表述。

也可以说 s1, s2 生命周期不一致，我们返回值的生命周期是比较小的那个。这是这段代码的表述。

2 1



老裴
2021-09-19

用其他语言写函数也会遇到生命周期的事，比如某某引用已经失效还是未初始化什么，标出来就清楚了，但是有没这种情况，某个函数的参数生命周期是变化的，可能是<'a,'a>也可能是<'a,'b>,这样的话函数要实现2遍吗

展开



核桃
2021-09-17

老师你好，这里有一些小疑惑没有搞懂。
1.&mut &str 添加生命周期后变成 &'b mut &'a str，这将导致返回的 '&str 无法选择一个合适的生命周期。

这句话里面,为什么mut前面还会需要标注生命周期的，这不是变量吧，因为在我故个人理解中，这是一个关键词和符号,为什么会需要生命周期标记？...

展开

作者回复: 对于 1，你可以再看看我文中画的图，好好感受一下 &mut &str 表示什么。&str 是一个字符串切片，&mut &str 是指向这个字符串切片的一个可变的引用。所以我们的问题是：返回值的生命周期究竟应该是和字符串切片有关？还是跟对这个字符串切片的可变引用有关？



Arthur

2021-09-15

1.
- 生命周期标注为`pub fn strtok<'a>(s: &'a mut &str, delimiter: char) -> &'a str `
- 错误信息为
```
error[E0502]: cannot borrow `s1` as immutable because it is also borrowed as mu...
```
展开

作者回复: 1. 正确！
2. 好问题，理论上可以，但编译器目前没有做这样的事情，主要是代码可读性的担忧。可以看：https://github.com/rust-lang/rfcs/blob/master/text/0141-lifetime-elision.md#lifetime-elision-in-structs。



记事本

2021-09-15

```
fn max<'a>(s1: &'a str, s2: &'a str) -> &'a str {
    if s1 > s2 {
   return s1
    } else {...
```
展开

作者回复: 不用加 return。这里没有提前返回， if {} else {} 是一个完整的表达式

2021-09-15

这是因为编译器希望尽可能减轻开发者的负担，其实所有使用了引用的函数，都需要生命周期的标注，只不过编译器会自动做这件事，省却了开发者的麻烦。

这个编译器会自动添加标注，有没有什么插件或者方法这个自动能变成显式的添加？

展开

作者回复: 没有。你可以把所有函数接口里没有标注的引用当做 'a 标注，方法里没有标注的引用看成 &'a self，返回值也是 &'a。

Tyler

2021-09-14

1、生命周期标注一个疑问，文中说生命周期标注是表明，参数与参数之前，参数与返回值之间的关系，那么函数名与参数列表中间那个 <'a> 就是下面代码中第一个<'a> 即: strtok<'a> 是代表的什么？ 有什么作用？ 与参数的标注，返回值的标注有什么关系？

```rust...
```

展开

作者回复: 1. <'a> 类似于一个泛型参数，它是说这个函数有几个类型参数。比如：

```rust
fn process<'a, 'b, T1, T1>(s1: &'a T1, s2: &'b T2) {...}
```

这个函数有两个生命周期参数 'a/'b，两个类型参数 T1/T2。

罗杰

2021-09-14

参考资料 5 有点疑惑了，RFC2094 - Non-lexical lifetimes 里面展示说在 None arm 里面执行 map.insert 是错误的

展开

作者回复: 在没有 NLL 之前，是错误的。NLL 优化了这类问题。



有铭
2021-09-14

看了几遍，还是未能理解，生命周期参数中的这个 a 啊 b 啊，是怎么来的，一定要从a开始吗？我标两个b上去行不行，有个例子里是一个a和一个b，那变成一个b和一个c行吗？

作者回复: 可以，你可以用任何你觉得容易明白的小写字母甚至单词，比如 'hello。就跟泛型的 T 一样，你也可以不用 Vec<T>，用 Vec<MyGenericType>。它就是一个名称。

1



亚伦碎语
2021-09-13

会发生所有权的冲突，不满足一个值可以有唯一一个活跃的可变引用。可变引用（写）和只读引用（读）是互斥的关系。
原因是因为让 s1的mut引用和只读引用有一样的scope，就会违反上述的规则，&mut &'a str则是s1的可变引用的scope只在strtok内。所以不违法上边的规则

展开

作者回复: 👍



码生笔谈
2021-09-13

老师这句话可能笔误了：“Employee 的生命周期不能小于它们”，
应该是”Employee 的生命周期需要小于它们“吧，因为文中的下一句话说的是：
“使用数据结构时，数据结构自身的生命周期，需要小于等于其内部字段的所有引用的生命周期。”

展开

编辑回复: 恩学习非常细致呀，棒👍之前有个同学也反馈了这个问题，已经修改过来啦